Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 7&8

Student Name: K.Saranya

Roll No.: A24126552090

Title

Develop a Mini Student Management Application using Node.js and Express.js

Aim

To design and develop a mini Student Management Application using Node.js and Express.js that provides REST API endpoints for retrieving and adding student information and presents the data through a simple HTML and CSS interface.

OBJECTIVES

- To understand the fundamentals of Node.js and Express.js.
- To create an Express.js server and handle HTTP requests.
- To create REST API endpoints using GET and POST methods.
- To store student information using JavaScript objects and arrays.
- To serve static HTML and CSS files using Express.
- To use express.json() to process JSON request data.
- To use fetch() in client-side JavaScript to communicate with the Express API.
- To dynamically display student information in an HTML table.
- To implement a simple form for adding new student records.
- To test the application using different input and API test cases.

Theory

Node.js is a JavaScript runtime environment that allows JavaScript code to execute outside the browser. It is commonly used for developing server-side applications. Express.js is a lightweight web framework built on Node.js that simplifies the creation of servers, routes, and APIs.

In this experiment, an Express server is created to manage student information. The student records are stored in an array of JavaScript objects. The GET /api/students endpoint returns all student records in JSON format, while the POST /api/students endpoint receives a new student record and adds it to the array. Express static middleware is used to serve the HTML and CSS files from the public folder.

The frontend uses HTML to create the student form and table, CSS to provide a suitable user interface, and JavaScript fetch() to communicate with the Express API. When the page loads, the GET API is called and the returned records are displayed dynamically. When a new student is submitted, a POST request sends the data to the server and the table is refreshed.

Concepts Used

Concept	Purpose
Node.js	Runs JavaScript on the server side.
Express.js	Creates the web server and API routes.
HTTP GET	Retrieves all student records.
HTTP POST	Adds a new student record.
express.json()	Parses JSON data received in requests.
express.static()	Serves frontend files from the public folder.
JavaScript Array	Stores multiple student objects.
Object	Stores the details of an individual student.
fetch()	Sends requests from the frontend to the Express API.
DOM Manipulation	Updates the student table dynamically.
JSON	Transfers student data between frontend and backend.

ALGORITHM / PROCEDURE

1. Create a Node.js project and initialize it using npm init -y.
2. Install Express.js using npm install express.
3. Create server.js and import the Express module.
4. Create an Express application and define the server port.
5. Enable express.json() to process JSON request bodies.
6. Create an array containing sample student objects.
7. Create a GET /api/students route to return all students.
8. Create a POST /api/students route to add a new student.
9. Create a public folder containing index.html and style.css.
10. Use express.static("public") to serve the frontend files.
11. Create a form in HTML to enter student name, age, and course.
12. Create a table to display registered students.
13. Create script.js and use fetch() to request student data from the API.
14. Display the returned students dynamically in the table.
15. Send form data to the POST API as JSON.
16. Refresh the student list after a successful addition.
17. Run the application using node server.js.
18. Open http://localhost:5000 in a browser and verify the output.
19. Perform multiple test cases and compare expected and actual results.

CODE

Server-side Code: server.js

```
const express = require("express");

const app = express();

const PORT = 5000;

app.use(express.json());
app.use(express.static("public"));

const students = [
  {
    id: 1,
    name: "Vani",
    age: 19,
    course: "CSE"
  },
  {
    id: 2,
    name: "Rahul",
    age: 20,
    course: "ECE"
  },
  {
    id: 3,
    name: "Priya",
    age: 19,
    course: "AI & ML"
  }
];

app.get("/api/students", (req, res) => {
  res.json(students);
});

app.post("/api/students", (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name,
    age: req.body.age,
    course: req.body.course
  };

  students.push(newStudent);

  res.status(201).json(newStudent);
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Frontend Code: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Management System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h1>Student Management System</h1>

    <div class="form-container">
      <h2>Add Student</h2>

      <form id="studentForm">
        <div class="form-group">
          <label for="name">Student Name</label>
          <input type="text" id="name"
                 placeholder="Enter student name" required>
        </div>
```

```
        <div class="form-group">
          <label for="age">Age</label>
          <input type="number" id="age"
                 placeholder="Enter age" required>
        </div>

        <div class="form-group">
          <label for="course">Course</label>
          <input type="text" id="course"
                 placeholder="Enter course" required>
        </div>

        <button type="submit">Add Student</button>
      </form>
    </div>

    <div class="student-container">
      <h2>Registered Students</h2>

      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Age</th>
            <th>Course</th>
          </tr>
        </thead>

        <tbody id="studentTable">
        </tbody>
      </table>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

Frontend Styling: style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f2f4f7;
  color: #333;
}

.container {
  width: 90%;
  max-width: 900px;
  margin: 40px auto;
}

h1 {
  text-align: center;
  color: #222;
  margin-bottom: 30px;
}

.form-container,
.student-container {
  background-color: white;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
  margin-bottom: 30px;
}

.form-group {
  margin-bottom: 18px;
}

label {
  display: block;
  margin-bottom: 7px;
  font-weight: bold;
}

input {
  width: 100%;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 16px;
}

button {
  width: 100%;
  padding: 12px;
  border: none;
  border-radius: 5px;
  background-color: #007bff;
  color: white;
  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background-color: #0056b3;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}

th {
  background-color: #007bff;
  color: white;
  padding: 12px;
  text-align: left;
}
```

```
td {
  padding: 12px;
  border-bottom: 1px solid #ddd;
}

tr:hover {
  background-color: #f5f5f5;
}
```

Client-side Code: script.js

```
const studentForm = document.getElementById("studentForm");
const studentTable = document.getElementById("studentTable");

// Get all students
function loadStudents() {
  fetch("/api/students")
    .then((response) => response.json())
    .then((students) => {
      studentTable.innerHTML = "";

      students.forEach((student) => {
        const row = document.createElement("tr");

        row.innerHTML = `
          <td>${student.id}</td>
          <td>${student.name}</td>
          <td>${student.age}</td>
          <td>${student.course}</td>
        `;

        studentTable.appendChild(row);
      });
    })
    .catch((error) => {
      console.log("Error loading students:", error);
    });
}

// Add a new student
studentForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const name = document.getElementById("name").value;
  const age = document.getElementById("age").value;
  const course = document.getElementById("course").value;

  const studentData = {
    name: name,
    age: age,
    course: course
  };

  fetch("/api/students", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(studentData)
  })
    .then((response) => response.json())
    .then(() => {
      alert("Student added successfully!");
      studentForm.reset();
      loadStudents();
    })
    .catch((error) => {
      console.log("Error adding student:", error);
    });
});

// Load students when page opens
loadStudents();
```

CODE EXPLANATION

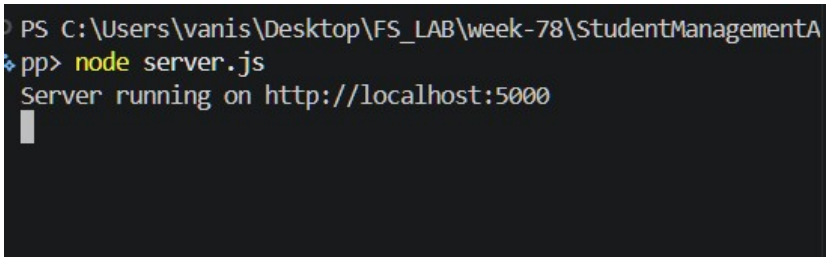
Code	Explanation
require("express")	Imports the Express.js framework.

Code	Explanation
express()	Creates the Express application.
app.use(express.json())	Allows the server to read JSON request bodies.
express.static("public")	Serves HTML, CSS, and JavaScript files from the public folder.
const students = []	Stores student records as JavaScript objects.
app.get("/api/students", ...)	Returns all student records in JSON format.
app.post("/api/students", ...)	Receives and adds a new student record.
req.body	Accesses student data sent in the POST request.
students.push()	Adds the new student object to the array.
res.json()	Sends JSON data back to the client.
res.status(201)	Indicates successful creation of a new resource.
fetch("/api/students")	Requests student information from the Express API.
forEach()	Iterates through all returned students.
createElement("tr")	Creates a table row dynamically.
innerHTML	Adds student information to the table row.
addEventListener("submit", ...)	Handles submission of the student form.

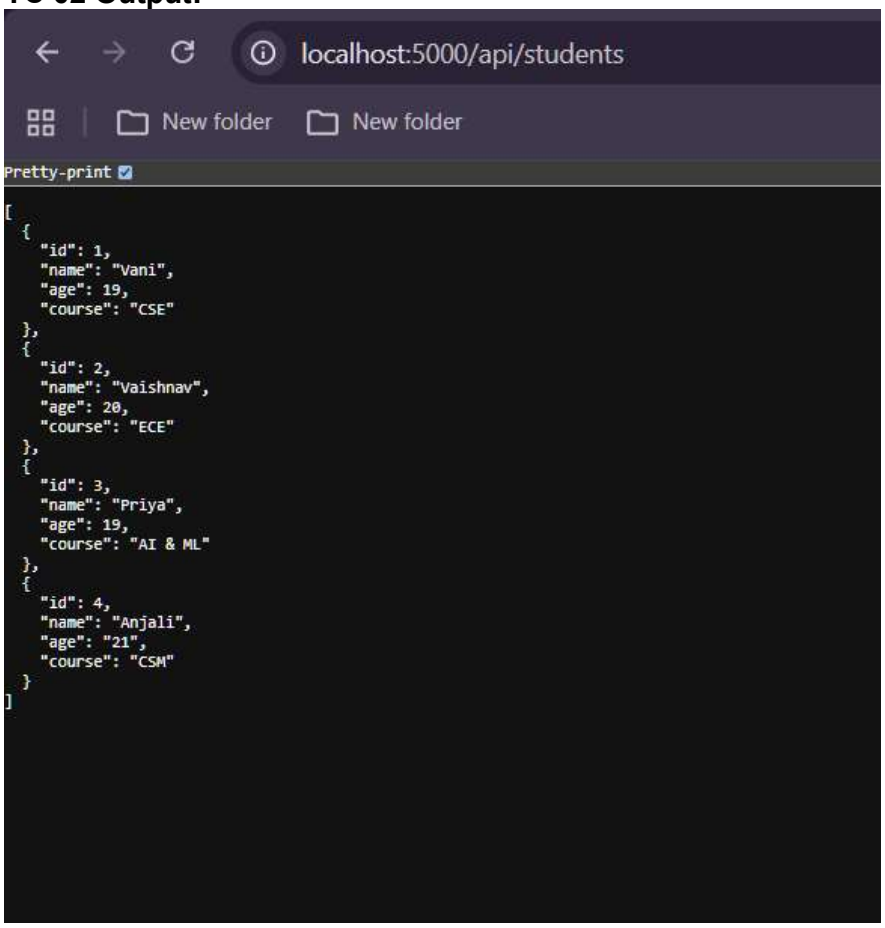
TEST CASES AND EXECUTION DATA

Test Case	Component Tested	Action / Input	Expected Result	Actual Result	Status
TC-01	Server	Run node server.js	Server should start on port 5000	Server started successfully	Pass
TC-02	GET API	Open /api/students	All student records should be returned	Student records displayed as JSON	Pass
TC-03	Frontend display	Open http://localhost:5000	Student form and table should be displayed	Form and student table displayed	Pass
TC-04	Add Student	Enter Anjali, 20, CSE and click Add Student	New student should be added and displayed	Student added and table refreshed	Pass
TC-05	Required validation	Submit form with an empty field	Browser should prevent incomplete submission	Required field validation displayed	Pass

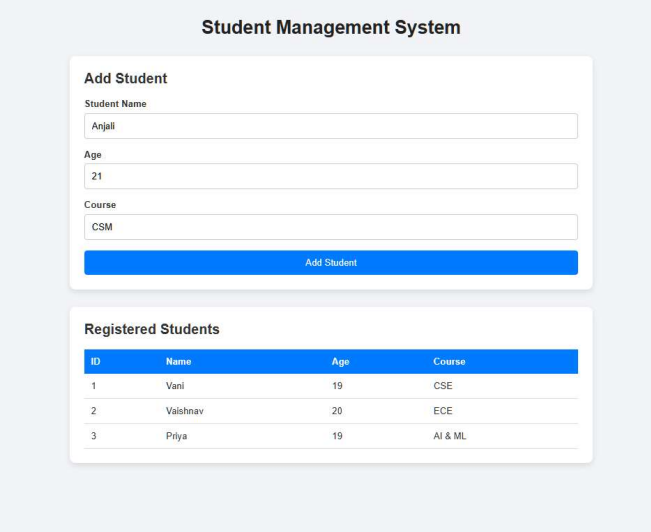
TC 01 Output:

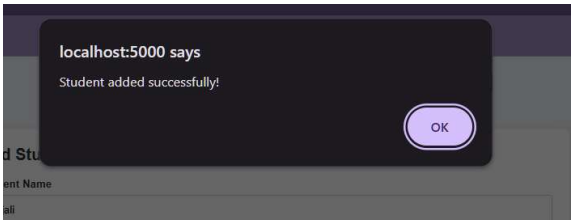


TC 02 Output:

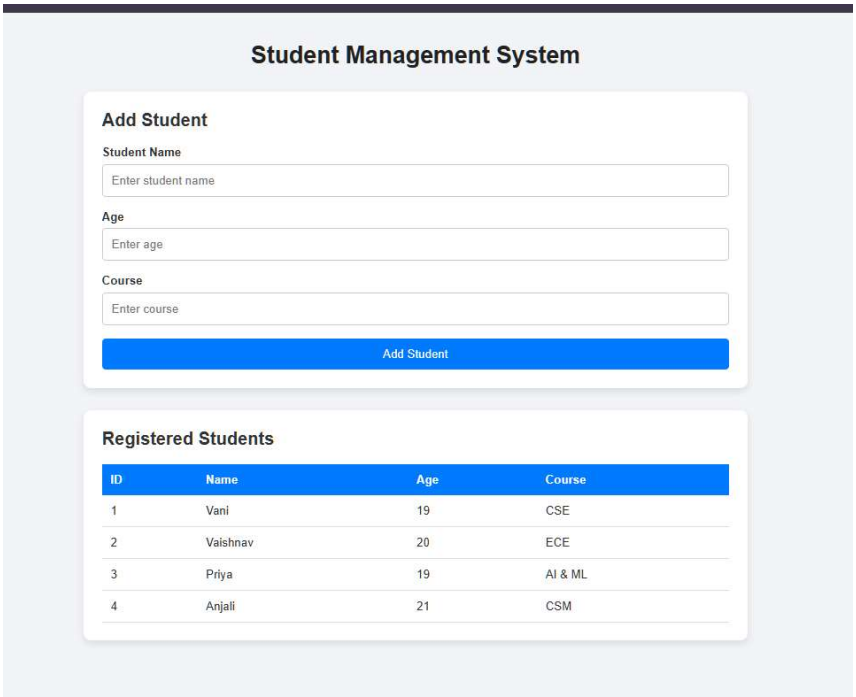


TC 04 Output :





OUTPUT / EXECUTION



When the application is executed using node server.js and opened at http://localhost:5000, the Student Management System page is displayed. The page contains an Add Student form and a Registered Students table. The initial sample records are loaded through the GET API. After entering a new student's name, age, and course, the POST API adds the record and the updated student list is displayed.

RESULT

Thus, a Mini Student Management Application was successfully developed using Node.js and Express.js. The application successfully creates an Express server, serves static frontend files, provides GET and POST REST APIs, stores student records, accepts new student data, and displays the records dynamically using HTML, CSS, and JavaScript. The application was tested using multiple test cases and the expected results were verified successfully.